

DEEP RESEARCH REPORT

Why Large-Scale Software Rearchitectures Succeed or Fail

An evidence-based analysis of diagnosis, migration, and organizational constraints

Prepared for: Engineering leadership review

Research basis: Company engineering blogs, conference talks, academic research, postmortems, interviews

Includes: the hypothesis and evidence, failure mechanisms, complexity displacement, Conway's Law, case studies and counterexamples, big-bang vs. incremental, a practical evaluation framework, and direct answers to 12 questions.

Why Large-Scale Software Rearchitectures Succeed or Fail

An evidence-based research report on diagnosis, migration, and organizational constraints.

Research date: August 27, 2026

Method: Multi-agent deep research — 5 parallel workstreams (failed rearchitectures, successful rearchitectures, conceptual mechanisms, organizational dynamics, enterprise/legacy programs). Sources prioritized: company engineering blogs, conference talks, academic papers, postmortems, regulator/audit reports, and technical interviews. Every claim is labeled **[FACT]** (documented), **[INTERPRETATION]** (informed reading), or **[SYNTHESIS]** (this report's cross-source analysis), with an inline source link and a confidence rating.

1. Executive Summary

The hypothesis is broadly supported, with important qualifications. Across ~25 reconstructed cases spanning technology companies (Netflix, Amazon, Google, Meta, Uber, Airbnb-adjacent, Dropbox, SoundCloud, Segment, Shopify, GitHub, LinkedIn, Etsy, Slack, Twitter/X), financial institutions (TSB, RBS, CBA, Deutsche Bank), and large government programs (NHS NPfIT, VA EHRM, California Medi-Cal, UK Universal Credit, Canada Phoenix), the evidence strongly supports the claim that **rearchitecture programs often begin from correct problem identification and a technically defensible target, yet fail or under-deliver not because the architecture was wrong in principle, but because of implementation, migration, organizational, and scoping compromises under real-world constraints.**

However, the research refines the hypothesis in three ways:

1. **Complexity is *conserved* and *relocated*, not eliminated.** The most consistently documented mechanism is that a rearchitecture changes *where* essential complexity lives (from in-process coupling to distributed-systems complexity: network, consistency, observability, operations) rather than removing it. This is not a failure of individual cases; it is a structural property of the architecture change.[M9][F6]
2. **The most common failure is not "bad architecture" but a small set of recurring categories:** the dominant ones are **implementation compromise** (rushed, under-resourced, missing tooling), **migration failure** (old/new coexisting too long and never completing), **incomplete organizational adoption** (teams keep old behavior on the new architecture), and **scope mismatch** (the wrong granularity or wrong-size workload). "Bad architecture" and "bad diagnosis" are present but less central than the hypothesis implies. [F14][F1]
3. **Success is a repeatable pattern, not luck.** The counterexamples (Netflix's 7-year cloud migration, Amazon's API mandate, Google's Borg, Shopify's modular monolith, GitHub's continuous upgrades) converge on a small set of enabling conditions: **incremental/strangler-fig execution that keeps the system working, long horizons, strong sponsorship, clear organizational ownership, automated migration tooling, and high observability.** Notably, two "successes" (Shopify, LinkedIn) succeeded by *avoiding* distribution — directly supporting the hypothesis from the opposite direction.[S7]

Bottom line for an engineering leader: A rearchitecture is rarely justified by the architectural idea alone. Its success is determined by whether the organization can absorb the operational tax of the target, whether the migration scaffolding will be removed (it usually won't), whether the team structure is re-cut to match (it usually isn't), and whether value is delivered incrementally from the start. Most failures are predictable before the first line of new code is written.

2. The Hypothesis

Large-scale rearchitecture projects often begin by correctly identifying real architectural, operational, or organizational problems, and the proposed target architecture may be technically sound. However, when the architecture is implemented under real-world constraints — deadlines, business priorities, migration complexity, compatibility requirements, staffing limitations, incomplete adoption, and pressure to continue shipping product features — necessary compromises can cause the new architecture to reproduce existing problems in different forms or introduce new ones.

This research set out to test this hypothesis skeptically, seeking both supporting and contradicting evidence, and to determine **which of eight explanations** best fits documented examples:

1. Bad diagnosis (problem misunderstood)
2. Bad architecture (target design flawed)
3. Implementation compromise (reasonable architecture, constrained execution)
4. Migration failure (old/new coexisted too long; integration overwhelmed)
5. Incomplete organizational adoption (teams didn't change behavior)
6. Scope mismatch (too many problems at once, or wrong-size workload)
7. Changing business environment (priorities changed mid-flight)
8. Successful transformation (worked, and why)

3. What the Evidence Says

3.1 Which failure category dominates

Across the 13 failed/struggling tech cases and the 9+ enterprise/government cases, the distribution of dominant failure categories is:

Failure category	Representative cases	Relative frequency
Implementation compromise	Phoenix Pay, FBI VCF, TSB, Segment, small-org postmortems	Very high — nearly universal as a contributing factor
Migration failure (old/new coexisted too long)	SoundCloud, Twitter/X, Medi-Cal, Universal Credit, TSB's rented clone	Very high in strangler/parallel-run programs
Incomplete organizational adoption	NPfIT, VA EHRM, Medi-Cal counties, RBS, Uber, SoundCloud	Very high — the binding constraint in most enterprise cases
Scope mismatch	Segment, Dropbox, Prime Video, small-orgs, Etsy (avoided)	High in microservices-decomposition cases
Bad architecture	Uber (unconstrained proliferation), Google's "microservices backfire" C1–C5, Prime Video	Medium — usually diagnosed <i>by the practitioners themselves</i> post-hoc
Bad diagnosis	Netscape rewrite, FBI VCF, Phoenix	Medium — rewriting a working-but-imperfect system
Changing business environment	NPfIT, VA EHRM, Deutsche/Postbank, Universal Credit, German banks	High on multi-year programs
Successful transformation	Netflix, Amazon, Google Borg/Go, Shopify, GitHub, LinkedIn, Slack, Etsy, CBA	—

[SYNTHESIS, High confidence] The most common failure mechanism across all cases is a composite: **the rearchitecture displaced the original problem into a new layer (network/dependency/operations)**

without reducing total system complexity, while the organization systematically under-estimated the operational and organizational tax of the target architecture. Bad architecture in isolation is rarely the sole cause; it usually becomes bad *in combination with* missing tooling, missing ownership, and missing adoption. [F14]

3.2 The single most important empirical result

The conceptual and case evidence converge on one central finding:

A rearchitecture changes where complexity lives; it does not remove the essential complexity of the problem. The "clean" target architecture is usually trading one set of accidental complexities (in-process coupling, monolithic deploy contention) for another (network latency and failures, distributed consistency, observability, per-service operations, version-interaction bugs, frozen APIs). [M9][F6]

This is not opinion. It is the documented conclusion of practitioners at the highest scale. Google's own HotOS 2023 paper states that "a microservices-based architecture like this **often backfires**, introducing challenges that counteract the benefits," and that microservices "conflate logical boundaries with physical boundaries," citing that in a study of 100+ catastrophic failures of eight widely used systems, **two-thirds of failures were caused by interactions between multiple versions** of a system. [F10] Martin Fowler's *Microservice Premium* and *Monolith First* make the same point: "microservices introduce complexity on their own account," and "almost all the successful microservice stories have started with a monolith that got too big and was broken up." [M9]

4. Why Rearchitectures Fail

4.1 The eight failure mechanisms, with evidence

1. Bad diagnosis — the original problem was misunderstood.

The Netscape rewrite (1998–2001) is the canonical case: the codebase was described as "ugly," but Spolsky's point is that it "worked pretty darn well on an awful lot of real world computer systems," and the from-scratch rewrite (Gecko) produced a ~3-year feature and market gap, gifting the browser market to IE. Montulli, one of the original five engineers, confirmed the rewrite was a major reason he resigned. [F1] Phoenix Pay is a diagnostic failure: a payroll system's *essential* complexity (rules, unions, tax, transfers) was treated as automatable commodity, and the result was ~153,000 of 300,000 employees with pay problems. [F3]

Diagnostic failure tends to coexist with scope mismatch: the fix was surgery, not a transplant.

2. Bad architecture — the target design was flawed.

Uber's 2,200+ microservices is the cleanest case: unconstrained service proliferation produced "networked monoliths" where "services that appear to be independent all have to be deployed together," requiring root-cause hunts through ~50 services across 12 teams, and a half-life of 1.5 years per microservice ("migration hell"). [F7] Google's C1–C5 failure modes (performance, correctness, manageability, frozen APIs, non-atomic deploys) are the same diagnosis from the practitioners themselves. [F10] Prime Video is a scope case: a distributed/Step-Functions architecture applied to a highly-coupled, data-shuffling workload was so expensive that consolidating into a single process cut costs by ~90%. [F9]

3. Implementation compromise — reasonable architecture, constrained execution.

Phoenix Pay was rushed, under-tested, under-trained, and under-funded, with warnings from the Miramichi Pay Centre ignored; the AG called it "an incomprehensible failure of project management and oversight." [F3] Segment's microservices failed in part because "we lacked the proper tooling for testing and deploying the

microservices when bulk updates were needed," and 3 full-time engineers were consumed keeping the system alive.[F6] TSB's migration "changed code on the hoof" over the go-live weekend.[E1.1]

4. Migration failure — old/new coexisted too long.

SoundCloud's public-API strangler took ~8 years and only completed after an extended fallow period; teams bypassed the monolith and accessed its DB directly, creating a two-class system.[F5] Twitter's decomposition reached a "thinner monolith" steady state and stopped: "It ends up being quite difficult to shrink a monolith down to 0." [F4] This is the "strangler that stopped strangling."

5. Incomplete organizational adoption — teams keep old behavior.

This is the **dominant binding constraint in the enterprise cases**. The NHS NPfIT technology largely existed but only 22 of 220 trusts adopted it.[E2.1] California's Medi-Cal state/county systems ran in parallel and were never reconciled, producing ~\$4B in questionable claims.[E2.3] VA EHRM sites "were allowed to act as if they were independent," producing fragmentation.[E2.2] Uber's ownership blurred across teams. **A new architecture in place with teams doing old behavior fails on uptake, not engineering.**

6. Scope mismatch — too many problems, or wrong-size workload.

Segment (homogeneous, low-varying transforms — wrong workload for microservices), Dropbox (decomposition for small self-contained functions was unnecessary overhead; the SOA effort was abandoned after 1.5 years and pivoted to a managed platform), Prime Video (data-intensive pipeline), and the small-org postmortems ("we went from 4 ECS services + 12 Lambdas + API Gateway chaos to one App Runner service and sleep better now").[F6][F8][F9][F13]

7. Changing business environment — priorities changed mid-flight.

Multi-year programs reliably outlive their own business rationale. NPfIT (cancelled after ~£10-12.7B), VA EHRM (paused 2023, restarted 2026), Deutsche Bank (a strategic review recommending a retail spin-off happened mid-transformation, forcing the architecture to follow the reorg), Universal Credit (≥£130M of IT written off, requirements repeatedly reshaped), and multiple German banks abandoning core replacements after hundreds of millions in sunk costs.[E5]

8. Successful transformation — see Section 9.

4.2 The meta-pattern: displaced, then re-mediated

Across almost every case, the eventual problems were **new or displaced versions of the original ones, not the original problems solved outright** — and the organization then spent a second cycle re-solving the complexity:

- Uber solved the monolith by going microservices, then built DOMA (domains/gateways/layers) to re-impose structure on the resulting mess.
- Dropbox abandoned SOA and built Atlas (a managed platform) to get service-like isolation without per-team ops.
- Segment reverted to a monolith with an aggregator (Centrifuge).
- SoundCloud completed the strangler only after ~8 years.
- Phoenix moved to a phased replacement ("we do not want to do another big bang").

This "displaced, then re-mediated" cycle is the strongest confirmation of the hypothesis.[F14]

5. How Complexity Gets Moved Rather Than Removed

5.1 The mechanism

Brooks's distinction between **essential** and **accidental** complexity is the foundation: a rearchitecture that only changes *where* complexity lives — from in-code coupling to in-network consistency and operations — is relocating, not removing, it.[M7] The essential problem complexity is largely invariant under an architectural container change.

The concrete translation: a monolith encapsulates complexity in-process (shared memory, function calls, one deploy). A microservices architecture translates that same essential complexity into **distributed-systems complexity**: network latency and partial failure, distributed transactions and eventual consistency, observability across N services, service discovery, per-service operations. Fowler's *Microservice Premium* codifies this — "microservices introduce complexity on their own account" — and his *Monolith First* observes that almost all successful microservice stories started from a working monolith that was broken up, while greenfield microservice systems "ended up in serious trouble." [M9]

The **distributed monolith** is the failure mode: "an application that's deployed like a microservice but is built like a monolith" — "the heaviness and inflexibility of monoliths, the complexity of microservices, and few of the benefits of either." [M9] The primary cause is a **shared database**: services that share a DB "make services look independent but keep them chained." [M9]

5.2 The empirical caveats (evidence is genuinely mixed)

- **Performance:** Blinowski et al. (IEEE Access 2022) found microservices *degraded* throughput and response times versus the monolith at lower load, only becoming competitive at high concurrency — they are not intrinsically faster. [M9]
- **Technical debt:** Lenarduzzi et al. (JSS 2020) found an *initial spike* in technical debt during migration, then slower debt growth than the monolith in the long run — debt is relocated and re-governed, not removed. [M9]
- **Velocity:** Saarimäki et al. (Empirical Software Engineering 2025) found **no statistically significant difference** in velocity between microservice and monolithic projects, and that coordination cost rises discontinuously as team count grows. [O1]
- **Operations:** DORA survey data indicates orgs with **>50 microservices report significantly longer mean time-to-recovery** for incidents. [O1]

[SYNTHESIS, High confidence] The literature genuinely disagrees on whether microservices outperform monoliths; the honest answer is "it depends on scale, domain, and org maturity." But the *relocation* framing — complexity moves into a different container rather than disappearing — is the consensus view and is highly consistent with the case evidence.

6. Migration as the Hidden Architecture

6.1 The transitional architecture is often the real architecture

The migration process itself — facades, anti-corruption layers, dual-run/dual-write synchronization, data-migration scripts, versioned/parallel APIs — becomes a load-bearing part of the system that often outlives the migration. [M10] The most important architecture of a rearchitected system is frequently the compatibility/ACL/dual-run scaffolding, because **that scaffolding is what users, data, and external systems actually interface with.** [M10]

6.2 "Temporary" becomes permanent — documented cases

The pattern is not theoretical; it is documented across government, banking, and insurance:

- **California Medi-Cal:** state and county eligibility systems ran in parallel and were never reconciled; the "temporary" dual-run became the permanent operating state, producing ~\$4B in questionable claims.[E4.1]
- **UK Universal Credit:** the "temporary" parallel state (UC alongside six legacy benefits) persisted for ~13 years; ≥£130M of IT was written off along the way.[E4.2]
- **TSB's rented Lloyds "mirror":** a temporary bridging arrangement became the production system for 5 years at ~£100M/year, and the eventual big-bang cutover failed in part *because* the interim system was an uncontrolled bodge.[E4.3]
- **COBOL-to-Java translation:** firms often end up maintaining the generated/translated code rather than the original; one company reverted to COBOL after millions because the translated code was unmaintainable. [E4.4]
- **The systemic version:** IDC projects **40% of global banks will run "sidecar"/wrapper cores by 2026, and 70–80% by 2028** — the temporary compatibility state is becoming the permanent industry default *by design*. [E4.5]
- **IPv4/IPv6:** the transition produced permanent dual-stack and NAT/translation fixtures.[O8]

The reframe: a "successful" migration is one where the scaffolding is *removed*, not merely *built*. Unless removal is explicitly scheduled with an owner, an exit condition, and usage measurement, the default outcome is a permanent half-migrated state.[M10][O8]

6.3 Data migration is the hardest part

Across the strangler literature and enterprise cases, data migration (database decomposition, dual-write, consistency) is consistently the most difficult and most-deferred component.[O5] Hyrum's Law explains why: with enough consumers, every observable behavior of the data — not just the documented contract — is depended upon, so the implicit interface must be emulated, not removed.[M6]

7. Organizational Constraints and Conway's Law

7.1 Conway's Law is the backbone

"Organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations" (Conway, 1968).[M2] The empirical "mirroring hypothesis" is supported: a loosely-coupled organization produces more modular software.[O4] **The critical consequence for rearchitecture:** if the org structure is not changed, the new architecture will be pulled back toward the old communication structure. Rearchitecture is a **socio-technical** problem; org design and target architecture must change together (the **Inverse Conway Maneuver** and **Team Topologies**).[M2][M11]

But Conway's Law is a heuristic tendency, not deterministic. Fowler notes it does not meaningfully constrain small teams, and the Inverse Conway maneuver "isn't going to be an instant fix" for an existing rigid architecture.[M2] **Org alignment biases architecture; it does not guarantee it.**

7.2 The ownership and adoption gap

The single most consistent predictor of platform-led rearchitecture trouble is **a mismatch between who drives the migration and who owns the code**. Pete Hodgson (martinfowler.com): the product team "is often not especially motivated" because the migration "is part of a broader architectural initiative without actually getting a huge amount of value themselves." [O2] DORA 2024 quantifies it: a platform team **lacking a product**

mindset is associated with ~8% lower throughput and ~14% lower stability, while a well-run platform raises productivity.[O2]

Empirical work confirms team–service ownership alignment is "often imperfect" and — critically — **cohesion and coupling are independent dimensions**: a highly cohesive service can still require cross-team coordination, and a loosely-coupled system can have fragmented ownership.[O4] You cannot assume that building new services produces clean team ownership; you must actively re-cut it.

7.3 Incentives are the real reason teams don't migrate

Feature work is rewarded; migration work (whose payoff lands in other teams' codebases, years later) is not. DORA 2024 found **priority instability correlates with ~90% of teams experiencing productivity drops**. [O3] Migration fatigue (repeated incremental migrations consuming engineering resources) erodes willingness to take on *yet another* transition.[O3] The practitioner reality — "I'm worried they will all just say no and refuse to migrate" — is widely corroborated.[O3]

7.4 Institutional knowledge loss and mid-program churn

Legacy modernization most commonly fails by losing the engineers who understood the legacy mid-project. "Knowledge debt isn't recoverable once the people who held it have left. The code remains, but the interpretive layer disappears." [O6] Long-running programs are, by construction, exposed to both reorg churn and knowledge erosion; programs with a fixed multi-year horizon and a single champion are fragile. Mitigation is about *durable* knowledge — characterization tests, decision logs/ADRs, living documentation — not just staffing.[O6]

8. Case Studies (deep reconstructions)

8.1 TSB Bank — Proteo4UK (2018): the canonical big-bang disaster

- **Original system:** TSB, spun off from Lloyds in 2013, operated on a rented "mirror copy" of Lloyds' merged systems (~£100M/year) — itself a "bodge of many old systems" from the HBOS–Lloyds integration.
- **Identified problem:** TSB ran on a bloated, expensive system it did not control.
- **Proposed architecture:** Sabadell bought TSB (2015) and pushed a **big-bang single-weekend migration** of 5.4M customers onto its Proteo4UK core.
- **Implementation reality:** Deadline compressed to ~18 months; the team lacked control of the source systems (Lloyds was still the supplier); by March 2017 the system was "hardly even a prototype"; go-live was followed within hours by collapse — up to 1.9M customers locked out, money "disappearing," some seeing other customers' accounts, direct debits dated 81 years in the future. Code was "changed on the hoof."
- **Compromises:** single-event migration prioritized to save fees over a safer phased approach; leadership ceded technical control to the parent; warning signs ignored for a year.
- **Outcome:** ~£330M cost, £48.65M combined fines (FCA + PRA), ~80,000 customers defected, CEO resigned.
- **Lessons:** a hard cutover deadline was set *before requirements were defined*; big-bang migration of a legacy integration you don't fully control is existential risk; parent/subsidiary governance disconnect killed course-correction.
- **Classification:** migration failure (big-bang) + implementation compromise + governance failure. The temporary-arrangement-became-permanent dynamic (the 5-year rented clone) directly caused the failure.

- **Displaced vs. new:** original cost problem persisted and worsened; new problems were mass customer lockout, fines, and trust collapse. **[High confidence]**

8.2 SoundCloud — the 8-year strangler that stalled

- **Original system:** a Ruby on Rails monolith whose DB schema growth "exceeds our ability to support it."
- **Proposed:** microservices on the JVM, extraction projects.
- **Reality — the stall:** "As a strategy for decommissioning our monolith, this strategy **has not served us well.**" Investing in the new stack made the monolith harder to staff ("many engineers feel intimidated by it and do not feel a sense of ownership"); teams **bypassed the monolith and accessed its DB directly** because the new stack was faster — "our investment in making services easier to build has created the conditions for behaviors which make it harder to decommission our monolith."
- **Outcome:** the public-API strangler took ~8 years and only completed after a fallow period that added complexity, lost knowledge, and accumulated security issues. Their own lesson: "make sure to have a plan to complete the migration **before the knowledge is lost.**"
- **Classification:** migration failure (coexisted too long) + incomplete org adoption.
- **Displaced vs. new:** complexity displaced into API duplication/inconsistency and a two-stack culture; the DB-schema problem was only solved after 8 years. **[High confidence]**

8.3 Segment — 100+ microservices back to one monolith

- **Original problem:** head-of-line blocking when one destination's retries flooded a shared queue.
- **Proposed:** one microservice + one queue per destination (fault isolation).
- **Reality:** as destinations grew past 100, operational overhead grew linearly (repo/queue/deploy each); shared libraries diverged; **3 FTEs consumed just keeping it alive**; "velocity plummeted, defect rate exploded."
- **Outcome:** reverted to a monolith with an aggregator (Centrifuge); developer productivity improved substantially.
- **Classification:** scope mismatch + implementation compromise (missing tooling).
- **Lesson:** the microservice granularity was wrong for the workload — homogeneous, low-varying transforms, not independently-evolving domains. Original blocking problem solved, but at the cost of $N \times$ operational overhead that dwarfed the benefit. **[High confidence]**

8.4 Uber — 2,200+ microservices and the DOMA response

- **Reality:** ~50 services across 12 teams to trace one root cause; "networked monoliths"; ownership blurred; "there is no turning back."
- **Response:** DOMA — grouping 2,200 services into ~70 domains behind gateways — re-imposed structure; onboarding time reduced 25–50%; some teams cut integration time from 3 days to 3 hours. Half-life of a microservice: **1.5 years**.
- **Classification:** bad architecture (unconstrained proliferation) — fixed with *more* architecture.
- **Displaced vs. new:** original monolith coupling displaced into dependency-graph blindness and churn; re-mediated by re-centralizing coordination at a coarser grain. **[High confidence]**

8.5 Dropbox — the SOA pivot to a managed platform

- **Reality:** after over a year and a half, the SOA effort "exposed the flaws of the second milestone": as more teams/services entered the critical path, maintaining reliability became harder and "would only compound." They concluded that "blindly splitting up our monolith into services would have increased operational load," and built **Atlas**, a managed platform that keeps code as a monolith but gives teams service-like isolation.

- **Classification:** migration failure (aborted) + scope mismatch.
- **Lesson:** a deliberate "stop the rearchitecture" — the pragmatic middle delivered most of the benefit. **[High confidence]**

8.6 Amazon Prime Video — microservices → monolith (90% cost cut)

- **Reality:** the distributed/Step-Functions design's two most expensive operations were "the orchestration workflow and data passing between distributed components." Consolidating into a single process cut costs ~90%.
- **Classification:** bad architecture / scope mismatch for a data-intensive, tightly-coupled transform.
- **Lesson:** even at Amazon, a service split applied to a data-shuffling workload was the wrong call. **[Medium confidence — primary URL confirmed, facts corroborated by secondaries]**

8.7 Netflix cloud migration + Cosmos (success, strangler fig)

- **Original:** a monolith in vertical datacenters; a 2008 database corruption stopped DVD shipping for 3 days.
- **Proposed:** AWS cloud-native, microservices. **Rejected lift-and-shift because it "moves all the problems and limitations of the data center along with it."**
- **What made it work:** ~7-year timeline; incremental migration (customer-facing first, billing/data last); strong exec sponsorship; Chaos Monkey/observability; severe legacy limits.
- **Outcome:** 8× streaming members, three orders of magnitude viewing growth, global launch.
- **Cosmos** (video-processing rebuild) explicitly used the **strangler-fig pattern** — old and new systems ran side by side, traffic routed incrementally.
- **Classification:** success. Factors present: a, b, e, h, k. **[High confidence]**

8.8 Amazon's API/SOA mandate (≈2002) → AWS

- **What made it work:** a CEO-backed mandate with consequences ("anyone who doesn't do this will be fired"), **strict architectural governance** (no direct linking, no shared stores, only service interfaces), and **two-pizza team ownership**.
- **Lesson:** a hard, executive-backed governance mandate can force a monolith-to-services transition that technology-consensus alone never would. The decisive factor was **organizational**, not technical. **[Medium-High confidence — Yegge memo provenance]**

8.9 Shopify — the modular monolith (deliberately NOT microservices)

- **Reality:** "splitting a single monolithic application into a distributed system of services increases the overall complexity considerably," so Shopify only split for good reasons (storefront rendering, credit-card vaulting). It made the Rails monolith modular with a dedicated Architecture Patterns team, Packwerk for dependency enforcement, and each component explicitly owned by a team.
- **Lesson:** the most successful "rearchitecture" was to *improve the monolith's internal modularity* and avoid microservices — directly refuting the relocation hypothesis from the opposite direction. **[High confidence]**

8.10 GitHub — continuous Rails/Ruby upgrades

- **Reality:** "There are no 'big bang' migrations." Every Monday a scheduled Action bumps Rails to the latest main; upgrades dropped from months to under a week. Migration strategy is "enablement and not replacement."
- **Lesson:** making a painful recurring rearchitecture *continuous and automated* converts it from periodic big-bang risk into routine work. **[High confidence]**

8.11 Additional high-value cases (summarized)

- **Netscape rewrite** — canonical big-rewrite failure; displaced complexity + market gap.[F1]
- **FBI Virtual Case File** — \$170M abandoned; 5 CIOs and 14 managers; original pain persisted unchanged. [F2]
- **Canada Phoenix** — ~half of 300k employees mis-paid; "incomprehensible failure"; 607k+ errors.[F3]
- **NHS NPfIT** — ~£10-12.7B cancelled; only 22/220 trusts adopted.[E2.1]
- **VA EHRM** — \$10B→\$16B→~\$37B; an 11,000-order routing bug caused patient harm; paused 2023, restarted 2026.[E2.2]
- **LinkedIn Lambda-less** — removed an entire offline batch stack (a de-simplification success).[S10]
- **Slack** — kept the PHP monolith, carved out infrastructure + cellular architecture (a negative case: avoided big-bang).[F11]
- **Etsy** — deliberately kept PHP; "the worse technology you can ship today beat the better technology that stalls you."[F12]

9. Counterexamples: When Large Rearchitectures Worked

9.1 The convergence of success factors

Across the 11 documented successes, the most common success factors (with frequency):

1. **Incremental migration despite an ambitious target (strangler fig)** — present in ~8/11 (Netflix cloud + Cosmos, Facebook Mercurial, Shopify, GitHub, LinkedIn, Google Go).
2. **Strong executive/cultural sponsorship** — Amazon (CEO mandate), Netflix (cloud-first), Etsy (blameless CD culture), GitHub (enablement philosophy).
3. **Clear organizational ownership** — Amazon two-pizza teams, Shopify components, Etsy dev on-call.
4. **Automated migration/ops tooling** — GitHub (scheduled upgrade PRs), Etsy (Deployinator), Shopify (Packwerk), Google Go, LinkedIn (Samza).
5. **High observability + safe-rollout discipline** — Netflix (Chaos Monkey), Etsy (graph-everything), Facebook (beta on all engineers' machines), GitHub (progressive deploys), Google Borg.
6. **Severe legacy limitations** — a painful, genuinely binding constraint supplies motivation and license.
7. **Long timelines** — Netflix (7 years), Borg (a decade of evolution), Amazon (~2000–2006).

9.2 The critical qualification

Two "successes" succeeded by **avoiding** the fashionable migration:

- **Shopify** improved the monolith's modularity rather than going microservices.
- **LinkedIn** removed a Lambda layer (a *de*-simplification).

And one success (CBA core banking) was the *second* attempt after earlier ~\$1B painful overruns — it worked because of five dress rehearsals, realistic testing, and collapsed silos, an "eventually it worked" arc rather than a clean first try.[E1.3]

[SYNTHESIS] The counterexamples do not show that rearchitecture is easy. They show that the failures the hypothesis describes are **avoidable** when teams combine incremental strangler-style execution, long horizons, ownership, tooling, and observability — rather than big-bang rewrites that cut over all at once.

10. Big Bang vs. Incremental Transformation

10.1 The evidence strongly favors incremental — with a caveat

The evidence base is heavily weighted toward incremental/strangler-fig approaches, but much of it is practitioner/vendor data rather than independent peer-reviewed measurement. **There is essentially no rigorous, controlled, large-N comparison of incremental vs. big-bang outcomes** — this is a genuine evidence gap.[O7]

What the available evidence does show:

- **Big-bang rewrite failure rates** are commonly cited at **60–70% with 2–3× cost overrun** (practitioner-consensus figures, not peer-reviewed).[O5]
- **Strangler-fig enterprise projects** (Modernization Intel, n=29): **76% success rate**; projects extracting **<5% of monolith functionality in the first 90 days had ~92–94% failure**; failed projects had a median sunk cost of \$2.1M.[O5]
- **Early traction predicts success**: lack of early real-world extraction/velocity is one of the two strongest execution predictors (the other is unresolved data migration).[O7]

10.2 The honest answer to "are big rewrites inherently riskier?"

Not inherently, but structurally. A big-bang rewrite concentrates all cutover risk into a single high-stakes event that (a) suspends feature delivery for the whole duration, (b) forces a feature-parity requirement that the new system chronically fails to meet, and (c) requires a hard date that either forces an unprepared cutover or slips repeatedly, eroding leadership confidence. Incremental approaches distribute risk across many small delivery points, keep the system working, and let each slice earn its keep.[M1][O5]

But incremental is **necessary, not sufficient**. The strangler's own temporary layers can become permanent (Section 6), and decomposition can create a distributed monolith (Section 5). Incremental execution reduces *cutover* risk; it does not remove *architectural* complexity.

10.3 When a large rearchitecture IS the correct strategy

The counterexamples cluster around specific conditions where large transformation was clearly correct:

- **Severe, binding legacy limitation** (Netflix's 2008 outage; Google's scale wall; Facebook's SCM bottleneck) — the old system cannot grow.
- **An organizational change is the real goal** (Amazon's API mandate to enable the platform; the Inverse Conway maneuver).
- **The org can absorb the operational tax** (dedicated teams, platform-as-product, tooling, observability).
- **A long horizon is genuinely acceptable** (Netflix's 7 years; Borg's decade).

11. Warning Signs of an Over-Ambitious Rearchitecture

Synthesized from the evidence, these signals indicate a rearchitecture is becoming too ambitious or is headed toward a predictable failure:

1. **A hard cutover date set before requirements are defined** (TSB).
2. **Feature-parity is the success criterion** — the new system must replicate everything the legacy does before it can ship (Netscape, TSB).
3. **The org cannot absorb the operational tax** — product teams aren't staffed/paid to run the target (Dropbox, SoundCloud, small-orgs).

4. **Team structure is not being re-cut** — you're changing code but not ownership (Uber, Medi-Cal counties, NPfIT).
5. **Migration work has no champion with budget** — platform team owns the architecture but not the products or incentives (Hodgson's adoption gap).
6. **The "temporary" scaffolding has no removal plan** — no owner, no exit condition, no usage measurement (Medi-Cal, Universal Credit, the sidecar trend).
7. **No early real-world traction planned** — the first slice delivers <5% in ~90 days (Modernization Intel's strongest failure predictor).
8. **Data migration is deferred** — "we'll handle the data later" (the most consistent hidden failure).
9. **The scope tries to solve too many problems at once** — multiple simultaneous transformations, or a rewrite of a working-but-imperfect system where surgery suffices (Netscape, Segment).
10. **Single champion / single-horizon** — the program depends on one person or a fixed multi-year window that will outlive the business rationale (VA, NPfIT, Deutsche).
11. **Institutional knowledge holders are not retained** — the engineers who understand the legacy are reassigned or leaving (SoundCloud, PlayerZero's knowledge crisis).
12. **Fashion, not fit** — the migration is adopted because microservices are "what everyone does," without the scale/org maturity to pay the tax (small-org postmortems, Segment).

12. Framework for Evaluating a Proposed Rearchitecture

12.1 Score the proposal across these dimensions

For each dimension, assess the current program honestly. This mirrors the dimensions in the research brief and is grounded in the evidence above.

Dimension	What to ask	Evidence basis
Number of systems affected	Is this one system or the whole estate? Whole-estate rewrites are where scope mismatch lives.	Segment, TSB, NPfIT
Number of teams involved	Coordination cost grows super-linearly ($n(n-1)/2$); >3 teams with shared decisions is where it bites.	Brooks, Saarimäki 2025
Migration duration	Can the org sustain this horizon? Multi-year programs outlive business rationale.	Netflix 7y (worked); NPfIT 9y (failed); Borg decade (worked)
Compatibility requirements	How many consumers depend on implicit behavior? Hyrum's Law: the more consumers, the more the old contract must be emulated.	Hyrum's Law, SoundCloud, IPv6
Ability to run old/new simultaneously	Can you strangler-fig it (keep legacy in production, route incrementally)? If not, why not?	Netflix Cosmos, Azure strangler
Business feature pressure	Can feature development be reduced or paused? If not, the migration competes with features and loses.	DORA priority instability; TSB
Operational criticality	Is a failed cutover existential (payroll, banking, healthcare, payments)?	Phoenix, TSB, VA EHRM
Data migration complexity	Is data the hardest, most-deferred part? Plan it first, not last.	Azure strangler; Universal Credit
Reversibility	Can you roll back a step? If a step is irreversible, that's a red flag.	Strangler fig; CBA rehearsals

Dimension	What to ask	Evidence basis
Observability	Can you compare old/new outputs, trace across services, and detect drift?	Netflix, Facebook beta, GitHub
Staffing assumptions	Are dedicated teams funded? Do product teams have the capacity and incentive to operate the target?	Hodgson; Dropbox; DORA platform
Dependency count	How many cross-team dependencies must be sequenced?	Uber; SoundCloud extraction
Degree of organizational change required	Are you re-cutting team boundaries and ownership (Inverse Conway), or only changing code?	Conway; org-cohesion study

12.2 A simple risk model (heuristic)

Assess each dimension above as **Low** / **Medium** / **High risk**, then use this qualitative scoring:

- **0–2 High-risk, or any single "existential" dimension (operational criticality = critical AND migration = big-bang AND control of source = low):** the proposal is likely over-ambitious as scoped. Require a redesign toward incremental/strangler execution, or explicitly reject.
- **Any single "no" to the four non-negotiables** → do not proceed as scoped:
 1. Can you deliver value incrementally (keep the system working, first slice <90 days, <5% extraction threshold avoided)?
 2. Is team ownership being re-cut to match the target?
 3. Is migration/removal of scaffolding explicitly owned, budgeted, and dated?
 4. Are the institutional-knowledge holders retained and their knowledge encoded (tests, ADRs)?
- **Low risk on most dimensions + a binding legacy limitation + long-horizon acceptance:** a large rearchitecture can be justified.

12.3 The four non-negotiables

Based on the strongest, most-repeated success factors and the most consistent failure mechanisms, any rearchitecture proposal should satisfy all four:

1. **Incremental + reversible execution.** Keep the legacy working; route traffic slice by slice; every step reversible. If a big-bang is genuinely required (rare), treat it as an exception requiring the highest scrutiny — and rehearse it (CBA's five dress rehearsals).
2. **Ownership re-cut.** Re-cut team boundaries and assign clear ownership of every new component/service before starting. Ownership that "emerges" becomes the adoption gap.
3. **Scaffolding removal is a deliverable.** The migration's success metric is the *removal* of compatibility/dual-run layers, with an owner, budget, and date. If you can't name the exit, you haven't planned a migration.
4. **Knowledge durability.** Retain the legacy-knowledge holders and convert their knowledge into characterization tests and decision logs. Knowledge debt is unrecoverable once the people leave.

13. Practical Guidance for Engineering Leaders

1. **Diagnose before you design.** The first question is not "what's the target architecture" but "what problem are we actually solving, and is it architectural, operational, or organizational?" A rearchitecture rarely fixes an organizational problem (NPfIT, Medi-Cal) or an incentive problem (Hodgson).
2. **Treat rearchitecture as socio-technical.** If you're changing code but not team structure, Conway's Law will pull it back. Plan the org change (Inverse Conway) in parallel.

3. **Resist the second-system effect.** A rewrite of a working system is precisely the situation Brooks warned about. Scope ruthlessly; the target should solve the identified problem, not every deferred feature.
4. **Respect Chesterton's Fence and Hyrum's Law.** Map *why* the legacy behaves as it does before removing it. Every observable behavior is load-bearing to someone.
5. **Data first, not last.** Data migration is the hardest and most-deferred part; schedule it early, not as the final gate.
6. **Measure adoption, not construction.** A platform that "shipped" but wasn't adopted failed. Tie rewards to adoption and product outcomes, not to building the abstraction.
7. **Schedule the cleanup.** The most common outcome is a permanent half-migration. Name who removes the scaffolding, when, and how you'll know it's safe to remove.
8. **Deliver value in 90 days or reconsider.** Early real-world traction is the strongest execution predictor. If the first slice can't move real traffic in ~90 days, the program is at risk.
9. **Retain the knowledge holders.** Losing the people who understand the legacy is a top non-technical failure driver; encode their knowledge before it's gone.
10. **When in doubt, strangle it.** Incremental strangler-fig execution with strong compatibility layers, observability, and reversibility converts a dangerous leap into a series of low-risk steps — and can be paused when features need priority.

14. Conclusions — Direct Answers to the 12 Questions

1. Is the hypothesis broadly supported by available evidence?

Yes — broadly, and with qualification. The strongest and most consistent finding is that rearchitectures frequently start from correct problem identification and a defensible target, then fail or under-deliver due to implementation, migration, organizational, and scoping compromises. But the research refines it: the dominant failures are not "bad architecture" per se but **implementation compromise, migration failure, incomplete adoption, and scope mismatch** — plus the structural fact that complexity is relocated, not removed.

2. What are the most common mechanisms by which a rearchitecture merely relocates complexity?

(a) **Distributed-monolith relocation:** monolith coupling becomes network/consistency/ops complexity (Uber, Google C1–C5, Fowler's premium). (b) **Hyrum's-Law compatibility drag:** the old implicit interface must be emulated via ACLs/facades that persist. (c) **Migration scaffolding as architecture:** dual-run/dual-write/facade layers become permanent. (d) **Erosion:** the new architecture immediately begins accruing its own debt (Lehman; Lenarduzzi's upfront TD spike).

3. Which architectural compromises are most dangerous?

Skipping the shared-data boundary (shared database → distributed monolith), wrong service granularity (Segment), and treating a code-structuring decision as a deployment decision (Google's conflation of logical/physical boundaries). The most dangerous single compromise is **splitting without a clear data-ownership boundary**.

4. Which migration compromises are usually survivable?

Reversible, incremental steps with strong compatibility layers and observability (Netflix Cosmos, GitHub's continuous upgrades, strangler fig). Dual-running is survivable *if* removal is scheduled; it is fatal when it becomes permanent (Medi-Cal, Universal Credit, TSB's clone).

5. Are large rewrites inherently riskier than incremental transformations, or is that an oversimplification?

It is an oversimplification to say "big-bang always fails." The evidence shows big-bang concentrates risk (feature-parity stall, single cutover, hard dates), but **incremental is necessary, not sufficient** — it doesn't

remove architectural complexity, and its scaffolding can become permanent. The rigorous large-N comparison does not exist; the practitioner consensus and case evidence favor incremental with strong caveats.

6. How does organizational structure constrain architectural outcomes?

Powerfully, via Conway's Law and the mirroring hypothesis: architecture mirrors org communication structure. If team boundaries aren't re-cut, the new architecture erodes back toward the old one. Ownership fragmentation and the platform/product adoption gap are consistent, empirically-documented constraints.

7. How often does "temporary" migration architecture become permanent?

Default, not exception. Documented across Medi-Cal, Universal Credit, TSB's rented clone, COBOL translation, IPv4/IPv6, and the industry-wide shift to sidecar/wrapper cores (IDC: 40% of banks by 2026, 70–80% by 2028). Without explicit removal scheduling, temporary scaffolding becomes permanent by inertia.

8. What warning signs indicate a rearchitecture is becoming too ambitious?

Hard cutover before requirements are defined; feature-parity as the success criterion; org can't absorb the operational tax; team structure not re-cut; no scaffold-removal plan; no early traction planned; data migration deferred; scope solving too many problems at once; single champion/horizon; knowledge holders leaving; fashion-driven adoption. (Full list in Section 11.)

9. What characteristics predict successful large-scale architectural transformation?

Incremental/strangler execution that keeps the system working; long acceptable horizons; strong executive/cultural sponsorship; clear organizational ownership; automated migration tooling; high observability; severe binding legacy limitations; early real-world traction; retention and encoding of institutional knowledge.

10. When should an organization explicitly choose not to rearchitect?

When the problem is organizational or incentive-based, not architectural (NPfIT, Medi-Cal); when the workload doesn't justify distribution (Segment, Prime Video, small-orgs); when the org can't absorb the operational tax (Dropbox); when the monolith is working and modularization/infrastructure extraction suffices (Shopify, Slack, Etsy, GitHub). When in doubt, improving the monolith's internal modularity beats a replacement.

11. When is a major rearchitecture justified despite the risks?

When there is a severe, binding legacy limitation (Netflix's outage; Google's scale wall); when an organizational change is the genuine goal (Amazon's API mandate); when the org can absorb the operational tax and a long horizon is acceptable; and when it can be executed incrementally with the four non-negotiables (Section 12.3).

12. What practical principles should engineering leaders use when evaluating a proposed rearchitecture?

Diagnose before designing; treat it as socio-technical; resist the second-system effect; respect Chesterton's Fence and Hyrum's Law; plan data first; measure adoption, not construction; schedule the cleanup; deliver value in 90 days or reconsider; retain the knowledge holders; when in doubt, strangle it. (Section 13.)

15. Sources / Bibliography

Sources are grouped by the research workstream that produced them; all were retrieved during this research project. Claims are anchored to primary sources (engineering blogs, papers, conference talks, postmortems, regulator/audit reports) wherever possible; secondary sources are used for corroboration only and are noted where applicable.

Conceptual mechanisms

- Conway, M. (1968), *How Do Committees Invent?* https://www.melconway.com/Home/Committees_Paper.html
- Brooks, F. (1975), *The Mythical Man-Month* (second-system effect; Brooks's law). https://en.wikipedia.org/wiki/Second-system_effect
- Brooks, F. (1986), *No Silver Bullet*. https://en.wikipedia.org/wiki/No_Silver_Bullet
- Gall, J. (1975), *Systemantics*. [https://en.wikipedia.org/wiki/John_Gall_\(author\)](https://en.wikipedia.org/wiki/John_Gall_(author))
- Fowler, M., *StranglerFigApplication* (2004). <https://martinfowler.com/bliki/StranglerFigApplication.html>
- Fowler, M., *Monolith First / Microservice Premium / Microservice Trade-Offs / Anti-Corruption Layer / Technical Debt / Technical Debt Quadrant*. <https://martinfowler.com/bliki/MonolithFirst.html> etc.
- Fowler, M. & Lewis, J. (2014), *Microservices*. <https://martinfowler.com/articles/microservices.html>
- Newman, S., *Monolith to Microservices* (O'Reilly, 2019). <https://samnewman.io/books/monolith-to-microservices/>
- Spolsky, J. (2000), *Things You Should Never Do, Part I*. <https://www.joelonsoftware.com/2000/04/06/things-you-should-never-do-part-i/>
- Hyrum's Law. <https://www.hyrumslaw.com/>
- Winters, Manshreck, Wright, *Software Engineering at Google* (2020).
- Skelton & Pais, *Team Topologies* (2019). <https://teamtopologies.com>
- ThoughtWorks, *Inverse Conway Maneuver*. <https://www.thoughtworks.com/radar/techniques/inverse-conway-maneuver>
- Gremlin, *Is your microservice a distributed monolith?* <https://www.gremlin.com/blog/is-your-microservice-a-distributed-monolith>
- Richardson, C., *Shared database* (microservices.io). <https://microservices.io/patterns/data/shared-database.html>
- Lehman & Belady, *Laws of Software Evolution*. https://en.wikipedia.org/wiki/Lehman's_laws_of_software_evolution
- Cunningham, W., *The WyCash Portfolio Management System* (OOPSLA 1992). <http://c2.com/doc/oopsla92.html>
- Herold et al. (2020), *Drift and Erosion in Software Architecture*, ICSDM. <https://dl.acm.org/doi/10.1145/3404663.3404665>
- Azure Architecture Center, *Strangler Fig pattern*. <https://learn.microsoft.com/en-us/azure/architecture/patterns/strangler-fig>
- Google Cloud, *Dual Run architecture*. <https://docs.cloud.google.com/mainframe-dual-run/docs/dual-run-architecture>

Academic / empirical

- Blinowski, Ojdowska, Przybylek (2022), *Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation*, IEEE Access 10. <https://doi.org/10.1109/ACCESS.2022.3152803>
- Lenarduzzi, Lomio, Saarimäki, Taibi (2020), *Does migrating a monolithic system to microservices decrease the technical debt?*, JSS 169. <https://doi.org/10.1016/j.jss.2020.110710>
- Saucedo et al. (2025), *Migration of monolithic systems to microservices: A systematic mapping study*, IST. <https://doi.org/10.1016/j.infsof.2024.107590>
- Auer et al. (2021), *From monolithic systems to Microservices: An assessment framework*, IST.

- Taibi, Lenarduzzi, Pahl (2017), *Processes, Motivations, and Issues for Migrating to Microservices*, IEEE Cloud Computing.
- Saarimäki et al. (2025), *Does microservice adoption impact velocity?*, Empirical Software Engineering. <https://link.springer.com/article/10.1007/s10664-025-10673-7>
- *Organizational Cohesion in Microservice Architectures* (arXiv, incl. Spinnaker study). <https://arxiv.org/html/2606.16725v2>
- Killalea, T., *Velocity in Software Engineering*, ACM Queue (2019). <https://queue.acm.org/detail.cfm?id=3352692>

Case studies — failed / struggling rearchitectures

- Segment/Twilio — Noonan, *Goodbye Microservices* (2018). <https://www.twilio.com/en-us/blog/developers/best-practices/goodbye-microservices>
- Uber Engineering — Gluck, *Introducing Domain-Oriented Microservice Architecture* (2020). <https://www.uber.com/us/en/blog/microservice-architecture>
- Dropbox — Sanguansin & Shah, *Atlas: Our journey from a Python monolith to a managed platform* (2021). <https://dropbox.tech/infrastructure/atlas--our-journey-from-a-python-monolith-to-a-managed-platform>
- SoundCloud — Stuart, *Microservices and the monolith* (2016); O'Brien & Creixell, *The End of the Public API Strangler* (2022). <https://developers.soundcloud.com/blog/microservices-and-the-monolith/>
- Prime Video Tech, *Scaling up the Prime Video audio/video monitoring service... reducing costs by 90%* (2023). <https://www.primevideotech.com/video-streaming/scaling-up-the-prime-video-audio-video-monitoring-service-and-reducing-costs-by-90>
- Google — Ghemawat et al., *Towards Modern Development of Cloud Applications*, HotOS'23. <https://pages.cs.wisc.edu/~rgrandl/papers/ms.pdf>
- Allen C., *Breaking Up the Twitter Monolith* (2023). <https://www.allenc.com/2023/02/breaking-up-the-twitter-monolith/>
- High Scalability, *The Great Microservices vs Monolithic Apps Twitter Melee* (2014). <https://highscalability.com/blog/2014/7/28/the-great-microservices-vs-monolithic-apps-twitter-melee.html>
- FBI Virtual Case File — IEEE Spectrum *Who Killed the Virtual Case File?*; Wikipedia. <https://spectrum.ieee.org/who-killed-the-virtual-case-file>
- Canada Phoenix — Wikipedia; JPIA Princeton *Burnt by Phoenix*; Ottawa Citizen. <https://jpia.princeton.edu/news/burnt-by-phoenix-canadas-costly-lesson-public-financial-management>
- Slack — Demmer, *Scaling Slack* (InfoQ 2018); Slack Engineering, *Migration to a Cellular Architecture*. <https://www.infoq.com/presentations/slack-scalability-2018/>
- Etsy — High Scalability, *Developing Products in the Style of Etsy* (2015). <http://highscalability.com/blog/2015/6/1/developing-products-in-the-style-of-etsy.html>

Case studies — successful / counterexamples

- Netflix, *Completing the Netflix Cloud Migration*. <https://about.netflix.com/en/news/completing-the-netflix-cloud-migration>
- Netflix TechBlog, *The Netflix Cosmos Platform*. <https://netflixtechblog.com/the-netflix-cosmos-platform-35c14d9351ad>
- Amazon — Yegge memo; InfoQ, *Microservices and Amazon*. <https://courses.cs.washington.edu/courses/cse452/23wi/papers/yegge-platform-rant.html>

- Google — Verma et al., *Large-scale cluster management at Google with Borg* (EuroSys 2015). <https://research.google.com/pubs/archive/43438.pdf>
- Google — go.dev, *Core Data Solutions*; Splash 2012 talk. <https://go.dev/solutions/google/coredata>
- Facebook — *Scaling Mercurial at Facebook* (2014). <https://engineering.fb.com/2014/01/07/core-infra/scaling-mercurial-at-facebook/>
- Shopify — *Re-architecting the Shopify monolith*. <https://shopify.engineering/shopify-monolith>
- GitHub — *Building GitHub with Ruby and Rails*. <https://github.blog/engineering/architecture-optimization/building-github-with-ruby-and-rails/>
- LinkedIn — *Lambda to Lambda-less architecture*. <https://engineering.linkedin.com/blog/2020/lambda-to-lambda-less-architecture>
- Etsy — *How Etsy Ships Apps*; InfoQ. <https://www.etsy.com/codecraft/how-etsy-ships-apps>
- CBA — CBA Newsroom core banking cloud migration; Accenture case study. <https://www.commbank.com.au/articles/newsroom/2026/03/commonwealth-bank-core-banking-cloud-migration-case-study.html>

Enterprise / financial / government

- TSB — The Guardian (2018); BBC; The Register; Computer Weekly; datascience.salon. <https://www.theguardian.com/business/2018/apr/28/warning-signs-for-tsbs-it-meltdown-were-clear-a-year-ago-insider> ; <https://www.bbc.com/news/business-64036529>
- RBS/NatWest — Reuters; MoneySavingExpert; The Guardian (2015). <https://www.reuters.com/article/technology/rbs-says-600000-payment-transactions-hit-by-it-problem-idUSKBN0OX0VL/>
- NHS NPfIT — Commonwealth Fund; PubMed; IEEE Spectrum. <https://www.commonwealthfund.org/publications/newsletter-article/nhs-cancels-ps12-billion-it-program>
- VA EHRM — Nextgov/FCW; FedScoop; Federal News Network. <https://www.nextgov.com/modernization/2026/05/ehr-restart-was-phenomenal-despite-persistent-challenges-initial-sites-va-secretary-says/413712>
- California Medi-Cal — KFF Health News; KCRA; CA State Auditor 2018-603. <https://kffhealthnews.org/news/article/deloitte-run-medicare-systems-errors-cost-millions-take-years-to-fix/>
- Universal Credit — House of Commons Library; Computer Weekly; The Independent. <https://commonslibrary.parliament.uk/research-briefings/cbp-9984/>
- Deutsche Bank — Computer Weekly. <https://www.computerweekly.com/news/4500244734/SAP-core-banking-platform-will-ease-Deutsche-Bank-split-following-strategic-retail-review>
- IDC sidecar-core projections — nextfinancial. <https://nextfinancial.substack.com/p/the-150-billion-core-banking-replacement>
- Air France-KLM / Amadeus — Travel Agent Central; KLM Newsroom. <https://www.travelagentcentral.com/transportation/air-france-klm-complete-change-to-amadeus>

Organizational dynamics & execution

- DORA 2024 State of DevOps. <https://cloud.google.com/blog/products/devops-sre/announcing-the-2024-dora-report>
- Hodgson, P., *How platform teams get stuff done* (martinfowler.com, 2023). <https://martinfowler.com/articles/platform-teams-stuff-done.html>
- microservices.io, *Service per team; Strangler Fig application pattern*.
- Modernization Intel, *Strangler Fig pattern* (n=29 projects). <https://softwaremodernizationservices.com/insights/strangler-fig-pattern-example/>

- PlayerZero, *Legacy modernization institutional knowledge crisis*. <https://playerzero.ai/resources/legacy-application-modernization-institutional-knowledge-crisis>
- IPv6 Migration Study, Internet Governance Project. <https://www.internetgovernance.org/wp-content/uploads/IPv6-Migration-Study-final-report.pdf>
- Microsoft Azure, *Strangler Fig* (data migration as hardest part). <https://learn.microsoft.com/en-us/azure/architecture/patterns/strangler-fig>